

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

*** ◇ ***



BÁO CÁO BÀI TẬP LỚN
XỬ LÝ NGÔN NGỮ TỰ NHIÊN

ĐỀ TÀI

VIQA NEXUS

HỆ THỐNG HỎI ĐÁP TIẾNG VIỆT

CÓ DẪN NGUỒN, GIA SƯ SOCRATIC VÀ VÒNG LẶP PHẢN HỒI

NHÓM 15

Mã môn học:	NLP Summer 2026	
Giảng viên hướng dẫn:	Trần Hồng Việt	
Nhóm thực hiện:	Nhóm 15	
Sinh viên thực hiện:	Nguyễn Tùng Dương	24022306
	Tổng Đức Hồng Anh	24022258
	Lê Toàn	24022466
	Lê Ngọc Quý	23021677
Mốc trạng thái mã nguồn:	20 tháng 08 năm 2026	

Hà Nội, 2026

Lời cảm ơn

Nhóm thực hiện xin trân trọng cảm ơn giảng viên hướng dẫn Trần Hồng Việt đã cung cấp nền tảng kiến thức về biểu diễn văn bản, truy hồi thông tin, mô hình ngôn ngữ tiền huấn luyện và phương pháp đánh giá hệ thống. Các góp ý trong quá trình học giúp nhóm chuyển đề tài từ một bản minh họa Retriever–Reader thành một hệ thống có cấu hình, benchmark, cơ chế từ chối trả lời, truy vết nguồn và kiểm thử hồi quy.

Nhóm cũng ghi nhận đóng góp của cộng đồng mã nguồn mở đối với PhoBERT, Hugging Face Transformers, Sentence Transformers, FAISS, React, Three.js và các thư viện được sử dụng trong dự án. Báo cáo này mô tả đúng trạng thái mã nguồn tại ngày 20/08/2026; những kết quả còn chưa đạt hoặc chưa được xác minh được trình bày như giới hạn, không được diễn giải thành khả năng đã hoàn thiện.

Tóm tắt

VIQA Nexus là hệ thống hỏi đáp trích xuất tiếng Việt trên kho tài liệu đóng. Một câu hỏi được đưa qua Retriever để lấy các passage liên quan, Reader để sinh nhiều span ứng viên, rồi qua lớp phân tích ngữ nghĩa và chuỗi gate nhằm kiểm tra tính hợp lệ của span, biên câu trả lời, bằng chứng, chủ thể, quan hệ và kiểu câu hỏi. Hệ thống trả lời kèm passage nguồn; nếu không có ứng viên vượt gate thì trả về trạng thái no-answer thay vì tạo nội dung mới.

So với phiên bản báo cáo trước, mã nguồn hiện đã có bốn Retriever ở API (TF-IDF, BM25, Dense và Hybrid), hai Reader có thể nạp (PhoBERT và XLM-R), semantic policy v2, cơ chế tinh chỉnh span, fallback có kiểm soát, chuẩn hóa source title, giao diện chat responsive với nhân vật 3D Mari, chế độ Gia sư Socratic và vòng lặp phản hồi có người kiểm duyệt. Socratic không dùng mô hình sinh ngoài: câu hỏi gợi mở được tạo từ bằng chứng trong các passage đã chọn/truy hỏi, khử trùng lặp và có thể được kiểm tra lại bằng BM25. Phản hồi được lưu trong SQLite ở trạng thái chờ duyệt; model và corpus không tự cập nhật trong runtime.

Benchmark Retriever trên 7.301 câu hỏi test cho thấy Hybrid đứng đầu với MRR 0,7682 và Recall@10 là 0,9156. Reader PhoBERT đang được runtime sử dụng đạt Overall EM 13,87 và Overall F1 36,25 trên toàn bộ 3.814 mẫu validation, nhưng Answerable EM chỉ 0,79; checkpoint vì vậy vẫn mang trạng thái *experimental*, chưa đủ điều kiện khẳng định production-ready. Lần chạy kiểm thử ngày 20/08/2026 cho kết quả 178 passed, 10 failed; frontend build thành công nhưng phát cảnh báo bundle lớn. Báo cáo cũng chỉ ra checksum của semantic holdout đang lệch file lock và trang Evaluation còn đọc artifact Reader cũ. Hai điểm này làm kết quả semantic mang tính tạm thời và dashboard chưa phải nguồn số liệu chính thức.

Mã nguồn: https://github.com/duong158/QA_system

Từ khóa: Question Answering, tiếng Việt, Retriever–Reader, Hybrid Retrieval, PhoBERT, semantic gating, Socratic tutoring, human-in-the-loop.

Quy ước về bằng chứng và phạm vi

Báo cáo phân biệt ba loại bằng chứng:

- **Runtime:** hành vi đọc trực tiếp từ `backend/viqa_api.py`, `config/qa_pipeline.json`, `config/semantic_policy.json` và `config/socratic.json`.
- **Artifact đánh giá:** số liệu đã lưu trong `results/`; mỗi bảng nêu rõ tập dữ liệu và file nguồn.
- **Kiểm tra tại thời điểm viết:** `npm run build` và `python -m pytest -q` chạy ngày 20/08/2026.

Điểm Retriever, Reader và ranking trong API là tín hiệu xếp hạng, không phải xác suất câu trả lời đúng. Hai trường `confidence` và `answer_confidence` được giữ để tương thích nhưng hiện trả về null.

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Quy ước về bằng chứng và phạm vi	iii
1 Giới thiệu	1
1.1 Bài toán	1
1.2 Mục tiêu hiện tại	1
1.3 Đóng góp của phiên bản đang báo cáo	2
1.4 Phạm vi và điều không được tuyên bố	2
2 Cơ sở lý thuyết và tiêu chí thiết kế	3
2.1 Retriever–Reader	3
2.2 Truy hồi lexical, dense và hybrid	3
2.3 Reader trích xuất và no-answer	3
2.4 Đánh giá	4
2.5 Nguyên tắc an toàn dữ liệu	4
3 Dữ liệu và tiền xử lý	5
3.1 UIT-ViQuAD2.0 và corpus	5
3.2 Chunking có biên câu	6
3.3 Căn chỉnh span và tokenizer	6
3.4 Chuẩn hóa tiêu đề nguồn	6
4 Kiến trúc hệ thống hiện tại	7

4.1	Luồng tổng thể	7
4.2	Phân tách offline và online	8
4.3	Cấu hình QA đang hoạt động	9
5	Retriever	10
5.1	Bốn phương pháp được hỗ trợ	10
5.2	Chuẩn hóa điểm	10
5.3	Lý do BM25 vẫn là mặc định	11
6	Reader, candidate và semantic gating	12
6.1	Reader được nạp	12
6.2	Hiệu chỉnh quyết định Reader	12
6.3	Sinh và tinh chỉnh ứng viên	12
6.4	Question semantics	13
6.5	Chuỗi gate	13
6.6	Reranking hiện tại	14
6.7	Truy vết và giải thích	14
7	Chế độ Gia sư Socratic	15
7.1	Mục tiêu	15
7.2	Quy trình tạo gợi ý	15
7.3	Tích hợp frontend	16
7.4	Giới hạn	16
8	Vòng lặp phản hồi có người kiểm duyệt	17
8.1	Mô hình dữ liệu	17
8.2	Xác minh feedback	17
8.3	Phân loại điểm mù	17
8.4	Không tự học trong runtime	18
8.5	Trạng thái dữ liệu hiện tại	18
9	Backend API và frontend	19
9.1	Backend	19

9.2	Response QA	20
9.3	Frontend chat mới	20
9.4	Avatar và giọng nói	20
9.5	Evaluation và Knowledge Blind Spots	21
9.6	Cấu trúc mã nguồn	21
10	Thực nghiệm và trạng thái kiểm chứng	22
10.1	Benchmark Retriever	22
10.2	Đánh giá Reader PhoBERT	23
10.3	Semantic holdout và ablation	24
10.4	Latency semantic	26
10.5	Build và test tại thời điểm viết	26
10.6	Kiểm chứng source title	26
11	Thảo luận, giới hạn và lộ trình	27
11.1	Điểm mạnh hiện tại	27
11.2	Giới hạn quan trọng	27
11.3	Lộ trình ưu tiên	28
11.4	Kết luận	29
A	Hướng dẫn chạy và tái lập	30
A.1	Chạy hệ thống	30
A.2	Build và kiểm thử	30
A.3	Mẫu request QA	31
B	Mười câu hỏi demo	32
C	Ma trận truy vết báo cáo–mã nguồn	33

Danh mục hình

4.1	Kiến trúc runtime của VIQA Nexus	7
10.1	Recall@ k từ artifact Retriever hiện tại	23
10.2	Reader PhoBERT trên full validation; nguồn: validation metrics của checkpoint	24
10.3	Ablation semantic; kết quả tạm thời do checksum lock đang lệch	25

Danh mục bảng

3.1	Thống kê các split đã xử lý	5
3.2	Kiểm tra ánh xạ answer span; nguồn: <code>results/span_integrity_report.json</code>	6
4.1	Phạm vi các thành phần	8
4.2	Snapshot config/qa_pipeline.json	9
9.1	API contract chính	19
10.1	Kết quả Retriever trên test; giá trị lớn hơn tốt hơn, trừ thời gian	22
10.2	Reader checkpoint đang được cấu hình	23
10.3	Ablation trên candidate pool cố định của semantic holdout	25
10.4	Kết quả kiểm tra ngày 20/08/2026	26
11.1	Backlog đề xuất	28
C.1	Nguồn kiểm chứng cho các nội dung chính	33

Chapter 1

Giới thiệu

1.1 Bài toán

Question Answering (QA) hướng tới việc trả lời trực tiếp câu hỏi tự nhiên thay vì chỉ trả về danh sách tài liệu. Với kho tiếng Việt cỡ vừa, việc cho Reader đọc toàn bộ corpus là tốn kém; kiến trúc phổ biến là Retriever–Reader: Retriever thu hẹp không gian tìm kiếm, Reader trích xuất span từ các đoạn liên quan, sau đó hệ thống xếp hạng và kiểm tra bằng chứng [1].

VIQA Nexus giải quyết biến thể *closed-domain extractive QA*. Câu trả lời hợp lệ phải là chuỗi có vị trí xác định trong passage nguồn. Hệ thống không hứa trả lời mọi câu hỏi; khả năng từ chối khi bằng chứng không đủ là một phần của đặc tả, đặc biệt với câu hỏi sai tiền đề hoặc câu hỏi có quan hệ ngữ nghĩa nhạy cảm như nguyên nhân, thời gian sinh/mất và địa điểm sự kiện.

1.2 Mục tiêu hiện tại

1. Xây dựng pipeline QA tiếng Việt có thể đổi Retriever và Reader qua cấu hình.
2. Bảo toàn liên kết từ câu trả lời đến document, passage và offset bằng chứng.
3. Kết hợp mô hình neural với luật ngữ nghĩa có version và gate có thể kiểm toán.
4. Cung cấp UI chat dễ dùng, hỗ trợ bàn phím, giọng nói và avatar Mari.
5. Gợi mở học tập bằng câu hỏi Socratic có nguồn, không tạo câu hỏi ngoài corpus.
6. Thu phản hồi theo vòng lặp có kiểm duyệt, không tự động học từ dữ liệu chưa duyệt.
7. Duy trì benchmark và test đủ để nhìn thấy cả cải tiến lẫn hồi quy.

1.3 Đóng góp của phiên bản đang báo cáo

- Online API hỗ trợ TF-IDF, BM25, Dense và Hybrid BM25+Dense bằng Reciprocal Rank Fusion (RRF) [3].
- Reader sinh tối đa năm span cho mỗi passage, kết hợp phrase fallback và sentence fallback, sau đó lọc bằng semantic policy v2.
- Parser tách subject, predicate, modifier và relation; validator chuyên biệt cho cause, location, time, purpose, identity, definition và contrast.
- Source title được ưu tiên theo metadata/heading, sau đó mới suy ra entity ở đầu passage; chi tiết ngày sinh—mất vẫn nằm trong evidence.
- Socratic follow-up và feedback/knowledge-blind-spot là hai lớp hậu xử lý độc lập, không làm hỏng đường trả lời chính nếu chúng gặp lỗi.
- Frontend được redesign thành hội thoại responsive, disclosure nguồn/pipeline, lịch sử, cài đặt, phản hồi, giọng nói và avatar 3D được lazy-load.

1.4 Phạm vi và điều không được tuyên bố

Hệ thống là prototype nghiên cứu chạy cục bộ, chưa có authentication/RBAC cho các API review, chưa tự động promote tài liệu hoặc checkpoint, chưa hiệu chuẩn xác suất đúng, và chưa chứng minh chất lượng trên một test set end-to-end có gold answer độc lập. XLM-R có checkpoint và có thể được API nạp, nhưng báo cáo hiện chỉ có artifact đánh giá đầy đủ cho PhoBERT. Pyserini có mã thử nghiệm offline nhưng không được nối vào online API.

Chapter 2

Cơ sở lý thuyết và tiêu chí thiết kế

2.1 Retriever–Reader

Với câu hỏi q và corpus $\mathcal{D}$, Retriever tạo tập passage ứng viên $\mathcal{P}_k(q)$. Reader ước lượng các span (s, e) trong từng passage. VIQA Nexus không chọn span có logit lớn nhất một cách trực tiếp mà sinh nhiều ứng viên, tính thêm tín hiệu kiểu câu hỏi và quan hệ rồi mới áp dụng gate cuối.

2.2 Truy hồi lexical, dense và hybrid

TF–IDF biểu diễn tài liệu bằng trọng số tần suất từ và nghịch đảo tần suất tài liệu. BM25 bổ sung bão hòa term frequency và chuẩn hóa độ dài [2]. Dense Retrieval mã hóa câu hỏi và tài liệu thành vector ngữ nghĩa; implementation offline dùng Sentence Transformers và FAISS, còn runtime dùng embedding đã chuẩn hóa và cosine similarity [4]. Hybrid lấy thứ hạng từ BM25 và Dense rồi hợp nhất bằng RRF:

$$\text{RRF}(d) = \frac{1}{K + r_{\text{BM25}}(d)} + \frac{1}{K + r_{\text{Dense}}(d)}, \quad K = 60. \quad (2.1)$$

2.3 Reader trích xuất và no-answer

PhoBERT kế thừa kiến trúc RoBERTa, được tiền huấn luyện cho tiếng Việt [6, 7]. Đầu QA dự đoán vị trí bắt đầu/kết thúc và null span. Runtime dùng margin

$$m = \text{score}(s^*, e^*) - \text{score}(\text{null}) \quad (2.2)$$

và threshold được hiệu chỉnh trên validation. Margin không phải xác suất; do đó báo cáo không gọi nó là confidence.

2.4 Đánh giá

Retriever được đo bằng Recall@ k và MRR. Reader được đo bằng Exact Match (EM), token F1, Answerable F1 và Unanswerable Accuracy. Với pipeline có từ chối, false positive là trả lời câu không nên trả lời; false negative là từ chối câu có đáp án. Hai loại lỗi này cần được trình bày cùng nhau vì tối ưu một phía có thể làm phía kia xấu đi.

2.5 Nguyên tắc an toàn dữ liệu

Vòng lặp phản hồi tuân theo hướng human-in-the-loop: dữ liệu người dùng phải được kiểm duyệt trước khi xuất thành tập huấn luyện [9]. Runtime chỉ ghi nhận phản hồi và phân tích điểm mù; không sửa trọng số model hoặc production corpus. Quyết định này hạn chế data poisoning và tránh biến một feedback đơn lẻ thành tri thức hệ thống.

Chapter 3

Dữ liệu và tiền xử lý

3.1 UIT-ViQuAD2.0 và corpus

Các file parquet đã xử lý có schema `id`, `title`, `context`, `question`, `answer_text`, `answer_start`. Thống kê đọc trực tiếp từ file tại thời điểm viết được trình bày ở [Table 3.1](#).

Table 3.1: Thống kê các split đã xử lý

Split	Tổng câu hỏi	Có đáp án	Không có đáp án/không công bố gold
Train	28.454	19.238	9.216
Validation	3.814	2.653	1.161
Test	7.301	0	7.301

Test parquet không chứa gold answer có thể dùng để chấm Reader, nhưng mỗi câu hỏi vẫn có context. Benchmark Retriever ánh xạ context đó về document trong corpus bằng exact match, substring hoặc Jaccard khi cần. Vì vậy kết quả test ở chương thực nghiệm là benchmark truy hồi document, không phải chất lượng trả lời end-to-end.

Corpus `data/processed/corpus_clean.parquet` và SQLite `data/processed/docs.db` có 5.317 document. Bảng documents trong SQLite hiện chỉ có hai cột `id` và `text`; không có title metadata. Khi API khởi động, 5.317 document được chia thành 6.544 passage theo cấu hình tối đa 220 token và chồng lấn hai câu.

3.2 Chunking có biên câu

Module backend/chunking.py tách paragraph, tách câu, rồi đóng gói các câu cho đến giới hạn token. Câu đơn quá dài được tách theo dấu chấm phẩy, hai chấm, dấu phẩy hoặc theo cửa sổ từ. Mỗi passage lưu document_id, passage_id, paragraph_id, sentence_start, sentence_end, nội dung và page nếu có. Hai câu overlap giúp giảm rủi ro đáp án nằm ngay biên chunk.

3.3 Cẩn chỉnh span và tokenizer

Pipeline Reader dùng raw text, PyVi một lần và tokenizer tương thích PhoBERT với offset tường minh. Báo cáo integrity hiện có kết quả ở [Table 3.2](#).

Table 3.2: Kiểm tra ánh xạ answer span; nguồn: results/span_integrity_report.json

Split	Mẫu có đáp án	Span hợp lệ	Span lỗi	Độ chính xác ánh xạ
Train	19.238	18.913	325	98,31%
Validation	2.653	2.614	39	98,53%

Phần lớn lỗi còn lại được gán nhãn TOKENIZER_BOUNDARY; ba mẫu train có đáp án dài hơn stride. Đây là giới hạn dữ liệu/tokenizer cần sửa trước khi kỳ vọng Answerable EM cao.

3.4 Chuẩn hóa tiêu đề nguồn

Tiêu đề không còn được lấy bằng cách cắt cứng câu đầu. Hàm derive_source_title ưu tiên: (1) document title; (2) heading; (3) entity chính ở đầu passage; (4) first sentence truncated. Hàm chỉ bỏ ngoặc khi nội dung có dấu hiệu birth/death range, không xóa ngoặc hợp lệ như “Hội nghị Genève (1954)”. Ví dụ:

Nguồn: Phạm Văn Đồng

Evidence: Phạm Văn Đồng (1 tháng 3 năm 1906 – 29 tháng 4 năm 2000) là Thủ tướng đầu tiên...

Đây là sửa lỗi metadata/title extraction; CSS truncate chỉ còn vai trò trình bày.

Chapter 4

Kiến trúc hệ thống hiện tại

4.1 Luồng tổng thể

Figure 4.1 cho thấy đường QA chính và hai lớp hậu xử lý. Socratic và feedback được tách khỏi quyết định trả lời để lỗi của chúng không chặn API /api/ask.

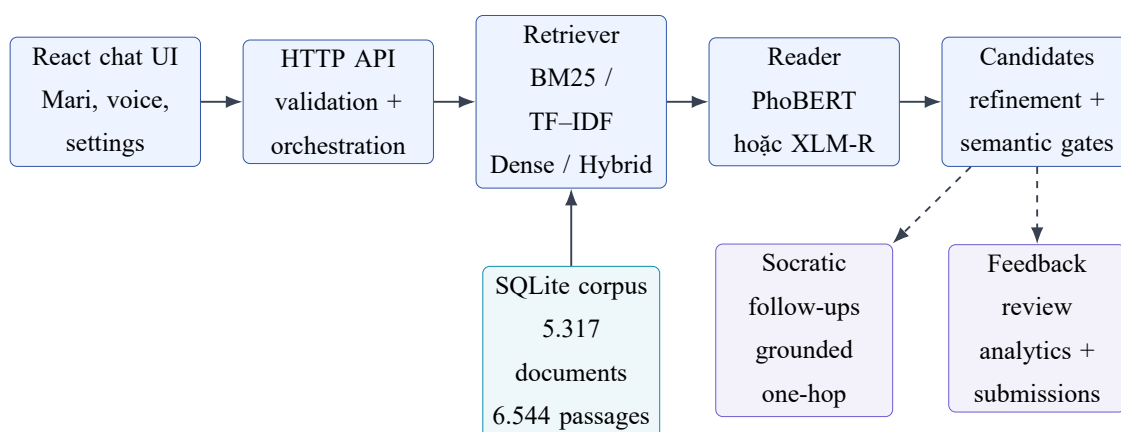


Figure 4.1: Kiến trúc runtime của VIQA Nexus

4.2 Phân tách offline và online

Table 4.1: Phạm vi các thành phần

Lớp	Offline	Online/runtime
Dữ liệu	clean parquet, build corpus/index, kiểm tra span	load SQLite, chunk passage khi khởi động
Retriever	module benchmark document-level, FAISS index	PassageIndex nội bộ, dense cosine, hybrid RRF
Reader	train/evaluate/calibrate checkpoint	lazy-load checkpoint, batch inference trên candidate passages
Semantic	build holdout, ablation, rule coverage	parse question, refine span, validate candidate, abstain
Feedback	export bản ghi approved thành JSONL	SQLite WAL, review queue, analytics; không học trực tiếp

Sự phân tách này rất quan trọng khi đọc số liệu: benchmark Retriever đang đo document-level index, trong khi API phục vụ passage-level index có thêm lexical boost. Kết quả offline mô tả xu hướng của phương pháp, không phải cam kết latency hay Recall giống hệ runtime.

4.3 Cấu hình QA đang hoạt động

Table 4.2: Snapshot config/qa_pipeline.json

Tham số	Giá trị	Ý nghĩa
default_retriever	BM25	lựa chọn mặc định của API/UI
default_top_k / max_top_k	10 / 20	passage trả về / trần request
Candidate pool	$\max(20, 2k)$, trần 40	số passage Reader phải đọc
Ranking weights	0,30 / 0,60 / 0,10 / 0,00	retriever / reader / answer type / relation
minimum_reader_score	0,30	gate tối thiểu của Reader signal
reader_span_candidates	5	số span neural tối đa mỗi passage
Reader window	256, stride 80	cửa sổ token và overlap
Chunking	220 token, overlap 2 câu	passage runtime
Checkpoint	heidiie PhoBERT (fine-tuned)	Reader PhoBERT đang nạp

Các trọng số phải không âm và tổng bằng 1; backend kiểm tra điều kiện khi load. Quan hệ có trọng số 0 trong phép cộng hiện tại nhưng vẫn có vai trò gate bắt buộc theo policy.

Chapter 5

Retriever

5.1 Bốn phương pháp được hỗ trợ

TF-IDF	Cosine trên vector TF-IDF; module benchmark dùng unigram/bigram, feature hashing kiểu DrQA và PyVi.
BM25	BM25 Okapi với $k_1 = 1,5$, $b = 0,75$. Online bổ sung query coverage, title match, n-gram proximity và lead-passage boost.
Dense	Mô hình keepitreal/vietnamese-sbert; offline dùng FAISS, runtime dùng cosine trên embedding chuẩn hóa được cache/precompute.
Hybrid	Lấy tối đa $3k$ ứng viên ở mỗi nhánh BM25 và Dense, hợp nhất bằng RRF rồi min-max normalize trong tập top- k .

Nếu Dense không thể nạp, online Dense/Hybrid hạ cấp về BM25 thay vì làm API dừng. Điều này tăng độ sẵn sàng nhưng response cần được đọc cùng health/config để biết phương pháp thực tế có hoạt động đầy đủ hay không.

5.2 Chuẩn hóa điểm

Điểm thô của các Retriever không cùng thang. API dùng min-max trong candidate pool:

$$\hat{s}_i = \begin{cases} \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j}, & \max_j s_j \neq \min_j s_j, \\ 1, & \max_j s_j = \min_j s_j > 0, \\ 0, & \text{ngược lại.} \end{cases} \quad (5.1)$$

Chuẩn hóa này thuận tiện cho reranking nhưng phụ thuộc candidate pool; không nên so trực tiếp một score runtime giữa hai câu hỏi khác nhau.

5.3 Lý do BM25 vẫn là mặc định

Hybrid tốt nhất trên benchmark hiện có, nhưng BM25 vẫn là mặc định vì model Dense làm tăng footprint và thời gian nạp ở host CPU, trong khi BM25 có đường fallback đơn giản và dễ chẩn đoán. Đây là lựa chọn vận hành, không phải kết luận BM25 chính xác hơn. UI cho phép chuyển TF-IDF, BM25, Dense và Hybrid; Pyserini được đánh dấu chưa triển khai online.

Chapter 6

Reader, candidate và semantic gating

6.1 Reader được nạp

API công bố phobert và xlmr. PhoBERT dùng checkpoint `models/reader/heidiie_phobert_finetuned_viquad`, nặng khoảng 513 MB; XLM-R dùng `models/reader/xlm-roberta-large-viquad`, nặng khoảng 2,08 GB. Reader được lazy-load và bộ predictor chỉ giữ model đang dùng để hạn chế bộ nhớ. UI có thể chọn XLM-R, nhưng artifact đánh giá chuẩn trong báo cáo chỉ dành cho PhoBERT.

6.2 Hiệu chỉnh quyết định Reader

File `models/reader/heidiie_phobert_finetuned_viquad/reader_profile.json` lưu threshold $-3,5$ trên margin best-span-minus-null. Profile đánh dấu calibrated vì được tạo từ toàn bộ 3.814 mẫu validation. Mục tiêu chọn threshold là

$$0,7 \times \text{AnswerableF1} + 0,3 \times \text{UnanswerableAccuracy}. \quad (6.1)$$

Tuy nhiên, “calibrated threshold” chỉ nói threshold có quy trình lựa chọn, không nói Reader đạt chất lượng production.

6.3 Sinh và tinh chỉnh ứng viên

Mỗi passage có thể đóng góp:

- tối đa năm `neural_span` từ Reader;

- `phrase_fallback` khi rule extractor tìm được cụm có bằng chứng;
- `sentence_fallback` như phương án cuối, chịu penalty 0,60 và gate kiểu câu hỏi nghiêm hơn (tối thiểu 0,75).

Span được kiểm tra offset, biên từ, bằng chứng substring; refinement có thể mở rộng hoặc rút gọn để tăng completeness nhưng vẫn phải giữ trực tiếp trong passage.

6.4 Question semantics

Parser trả về question type và cấu trúc semantic gồm subject, target, predicate, modifier, relation và trạng thái parse. Các policy chuyên biệt gồm CAUSE, BIRTH_TIME, DEATH_TIME, EVENT_LOCATION, OBJECT_LOCATION, PURPOSE, CONTRAST, DEFINITION, IDENTITY, ATTRIBUTE và GENERAL. Policy được version là v2.

6.5 Chuỗi gate

Ứng viên đi qua đúng thứ tự:

1. span hợp lệ;
2. boundary;
3. completeness;
4. evidence;
5. subject consistency;
6. relation validation;
7. answer-type compatibility;
8. final ranking.

Threshold chung là boundary 0,20; evidence 0,50; completeness 0,50; answer type 0,50 và ranking 0,60. Policy có thể yêu cầu subject/relation mạnh hơn; ví dụ birth và death time dùng minimum relation score 0,90. Một số quan hệ grounded được phép bypass ranking threshold nhưng không bypass bằng chứng hay các gate bắt buộc khác.

6.6 Reranking hiện tại

Với candidate c , Reader signal sau penalty là

$$R_c = s_{\text{reader}} p_{\text{fallback}} (0,5 + 0,5b_c) (0,5 + 0,5u_c), \quad (6.2)$$

trong đó b_c là boundary score và u_c là completeness score. Điểm cuối:

$$S_c = 0,30 s_{\text{ret}} + 0,60 R_c + 0,10 s_{\text{type}} + 0,00 s_{\text{relation}}. \quad (6.3)$$

Relation score vẫn quyết định pass/fail trước khi Equation (6.3) được dùng. Ứng viên pass gate có S_c lớn nhất được chọn; nếu không có ứng viên hợp lệ, API trả `has_answer=false`, source rỗng và rejection reason cụ thể.

6.7 Truy vết và giải thích

Response giữ raw span, refined span, offsets, candidate list, gate results, semantic parse, điểm thành phần và lý do rejection. Production UI chỉ hiển thị câu trả lời, nguồn và pipeline tóm tắt; chi tiết score chỉ hiện ở development/debug nhằm tránh tạo ấn tượng đây là xác suất chính xác.

Chapter 7

Chế độ Gia sư Socratic

7.1 Mục tiêu

Sau khi có câu trả lời, Gia sư Socratic gợi ý tối đa ba câu hỏi tiếp theo để người học khám phá quan hệ khác của cùng chủ thể. Module không thay thế QA chính và không gọi mô hình sinh ngoài. Câu gợi ý chỉ được tạo khi có subject và evidence trong selected/retrieved passages.

7.2 Quy trình tạo gợi ý

1. Lấy selected passage trước, sau đó thêm các passage truy hồi, tối đa 12.
2. Khôi phục surface form của subject từ corpus.
3. Phát hiện pattern trong từng câu cho birth/death time, location, role, event, cause, purpose, consequence, context, comparison và identity.
4. Loại relation vừa hỏi, relation đã thăm và câu quá giống lịch sử; ngưỡng similarity là 0,72.
5. Tính answerability; ngưỡng tối thiểu là 0,62. Passage xa có thể được BM25 probe top-5 xác minh.
6. Xếp hạng và lấy tối đa ba câu, chỉ một hop.

Điểm xếp hạng Socratic là

$$S_{\text{soc}} = 0,5A + 0,3R + 0,2N, \quad (7.1)$$

với A là answerability, R là relevance và N là novelty. Đây cũng là ranking signal, không phải xác suất câu tiếp theo chắc chắn có đáp án.

7.3 Tích hợp frontend

Hook useSocraticFollowups chỉ gọi `/api/socratic/followups` sau khi câu trả lời chính hoàn tất và toggle được bật. Khi người dùng chọn một gợi ý, câu đó trở thành user turn mới và đi qua cùng pipeline `/api/ask`. Session lưu relation đã thăm và câu đã hỏi để giảm lặp. Toggle xuất hiện ở header/ô soạn trên màn hình phù hợp; knob dùng kích thước cố định trong track và Enter/Shift+Enter được xử lý ở textarea.

7.4 Giới hạn

Rule-based generation tránh hallucination nhưng phụ thuộc parser và độ phủ pattern. Nếu subject parse sai, module trả danh sách rỗng thay vì đoán. Kết quả gợi ý chưa có benchmark người dùng chính thức về tính sư phạm; score hiện tại chỉ phản ánh bằng chứng, liên quan và độ mới trong corpus.

Chapter 8

Vòng lặp phản hồi có người kiểm duyệt

8.1 Mô hình dữ liệu

Feedback được lưu trong `data/feedback/feedback.db` bằng SQLite, WAL mode, busy timeout 5 giây và write lock trong process. Các loại phản hồi gồm `CORRECT`, `INCORRECT`, `SPAN_CORRECTION`, `NO_ANSWER_BUT_SHOULD_HAVE` và `ANSWERED_BUT_SHOULD_NOT`. Tài liệu đóng góp được lưu ở bảng riêng với loại `PLAIN_TEXT/TXT`.

Mỗi record giữ question, predicted answer, question type, relation, subject, selected passage, corrected passage/span, rejection reason, model/corpus/policy version, source của feedback, trạng thái review, cờ synthetic, conflict và duplicate count.

8.2 Xác minh feedback

Span correction phải trở đến passage tồn tại và corrected answer phải khớp substring theo offset. Fingerprint chuẩn hóa giúp deduplicate; phản hồi mâu thuẫn cho cùng ngữ cảnh được đánh dấu conflict thay vì âm thầm ghi đè. Lifecycle gồm `PENDING`, `REVIEWED`, `APPROVED`, `REJECTED`. Document submission có `PENDING_REVIEW`, `APPROVED`, `REJECTED`.

8.3 Phân loại điểm mù

Khi metadata đủ, hệ thống phân loại:

- `CORPUS_GAP`: corpus không có bằng chứng;
- `RETRIEVAL_GAP`: passage sửa đúng không nằm trong danh sách retrieved;

- `READER_SEMANTIC_GAP`: passage đúng đã được truy hồi nhưng chọn span/ relation sai;
- `UNKNOWN_GAP`: dữ liệu chưa đủ để phân loại.

Dashboard tổng hợp theo relation, question type, rejection reason, ngày và heatmap. Blind-spot score được định nghĩa:

$$B = \text{failure_rate} \times \ln(1 + n). \quad (8.1)$$

Duplicate count được dùng làm trọng số; dữ liệu synthetic được tách khỏi feedback thật.

8.4 Không tự học trong runtime

Các endpoint `submit/review` chỉ cập nhật kho review. Response ghi rõ `runtime_model_updated=false` hoặc `production_corpus_updated=false`. Chỉ record approved mới được export offline thành JSONL để kiểm tra, versioning, train và benchmark. Việc promote checkpoint/corpus phải là quyết định của con người.

8.5 Trạng thái dữ liệu hiện tại

Artifact `results/knowledge_blind_spot_report.json` tại thời điểm viết có 0 feedback. Vì vậy hệ thống đã có pipeline analytics nhưng chưa có cơ sở để kết luận relation hay question type nào là “điểm mù thực tế” từ người dùng. Dashboard trông là trạng thái đúng, không phải bằng chứng tỷ lệ lỗi bằng 0.

Chapter 9

Backend API và frontend

9.1 Backend

Backend là `ThreadingHTTPServer` trong Python, khởi động mặc định tại cổng 8000. Nó load corpus/passages, cấu hình policy, feedback store; Reader và Dense model được nạp lười. CORS và JSON response được xử lý trong một handler chung. Kiến trúc này phù hợp demo cục bộ nhưng chưa có schema framework/OpenAPI, worker queue hoặc readiness probe tách biệt.

Table 9.1: API contract chính

Endpoint	Method	Chức năng
/health	GET	passages, model, retriever/reader hỗ trợ, config, version
/api/ask	POST	chạy Retriever–Reader–semantic gate
/api/compare	POST	so top passage và latency; không có Recall nếu thiếu gold
/api/evaluation	GET	đọc artifact để phục vụ dashboard đánh giá
/api/socratic/followups	POST	sinh câu hỏi gợi mở có nguồn
/api/feedback	POST	ghi nhận phản hồi câu trả lời
/api/feedback/review	GET	danh sách feedback theo trạng thái
/api/feedback/:id/review	POST	review, approve hoặc reject feedback
/api/feedback/analytics	GET	summary, relation, type, gap, trend và heatmap

Endpoint	Method	Chức năng
/api/documents/submissions	GET/POST	liệt kê hoặc đóng góp tài liệu
/api/documents/submissions/:id/review	POST	duyet tài liệu

9.2 Response QA

Ngoài answer và has_answer, response chứa source, top retrieved passage, visible passages, selected candidate, raw/refined span, semantic parse, gates, score components, rejection reason, scoring config và system version. Frontend chuẩn hóa answer_source/source, default cho các field nullable và timeout request sau 60 giây.

9.3 Frontend chat mới

Frontend dùng React 18, TypeScript, Vite, Tailwind CSS, Zustand và React Router. Giao diện chính có ba vùng thích ứng:

- sidebar lịch sử, tạo chat mới và đường dẫn Evaluation/Knowledge Blind Spots;
- conversation ở trung tâm, user/assistant turns, trạng thái “Mari đang suy nghĩ”, disclosure nguồn, đọc/sao chép, feedback và Socratic follow-ups;
- Mari 3D ở desktop từ 1200 px; mobile dùng sheet và chỉ nạp component khi cần.

Composer dùng textarea: Enter gửi, Shift+Enter xuống dòng, bỏ qua Enter khi IME đang composing. Input speech được hợp nhất với draft rồi reset khi người dùng sửa tay. Nút gửi bị khóa khi rỗng hoặc đang xử lý. Lịch sử và cài đặt được lưu bằng Zustand; history giới hạn 20 mục.

9.4 Avatar và giọng nói

Mari sử dụng React Three Fiber, Three.js và @pixiv/three-vrm; có các state listening, typing, retrieving, reading, thinking, speaking, success, no-answer và error. Speech Recognition/Synthesis dựa trên Web Speech API của trình duyệt. Voice, rate, pitch và volume có thể cấu hình. Đây là lớp trải nghiệm; chất lượng QA không phụ thuộc avatar hoặc speech.

9.5 Evaluation và Knowledge Blind Spots

Trang Evaluation gọi API thật thay vì mock frontend. Tuy nhiên endpoint hiện ghép `results/retriever_eval_all.json` với file Reader legacy `results/reader_eval_results.json`; latency 248 ms và một số error count còn là giá trị trình bày tĩnh. Vì vậy số Reader trong trang này có thể khác artifact checkpoint đang chạy. Báo cáo sử dụng trực tiếp `results/reader/heidiie_phobert_finetuned_viquad_full/validation_metrics.json` làm nguồn chính thức và coi việc thống nhất dashboard là việc cần sửa.

Trang Knowledge Blind Spots gọi `analytics/review` API thật, hiển thị failure theo relation/type, gap distribution, trend, heatmap, queue review và form đóng góp tài liệu. Các route review hiện chưa có authentication/RBAC nên chỉ phù hợp môi trường cục bộ đáng tin cậy.

9.6 Cấu trúc mã nguồn

```
QA_system/
|-- backend/      # API, chunking, source title, Socratic, feedback
|-- reader/       # Reader inference/train/eval, semantics, gates, refinement
|-- retrieval/    # TF-IDF, BM25, Dense, Pyserini, build/evaluate scripts
|-- src/         # React pages, components, hooks, services, store, types
|-- config/       # qa_pipeline.json, semantic_policy.json, socratic.json
|-- data/        # raw/processed/evaluation/feedback
|-- models/reader/ # PhoBERT and XLM-R checkpoints
|-- results/     # metrics, predictions, diagnostics, reports
|-- tests/       # unit, integration and regression tests
|-- IMG/        # figures used by this report
`-- report.tex
```

Chapter 10

Thực nghiệm và trạng thái kiểm chứng

10.1 Benchmark Retriever

Thiết lập. Artifact results/retriever_eval_all.json đánh giá bốn phương pháp trên 7.301 câu hỏi test, gold là document chứa context. Kết quả trong [Table 10.1](#) and [figure 10.1](#).

Table 10.1: Kết quả Retriever trên test; giá trị lớn hơn tốt hơn, trừ thời gian

Phương pháp	MRR	R@1	R@3	R@5	R@10	QPS	Thời gian (s)
BM25	0,6856	0,5946	0,7444	0,7970	0,8507	56,8	128,5
TF-IDF	0,6512	0,5582	0,7121	0,7634	0,8142	51,3	142,1
Dense	0,7321	0,6654	0,8012	0,8541	0,8923	213,5	34,2
Hybrid	0,7682	0,6981	0,8345	0,8812	0,9156	44,8	162,7

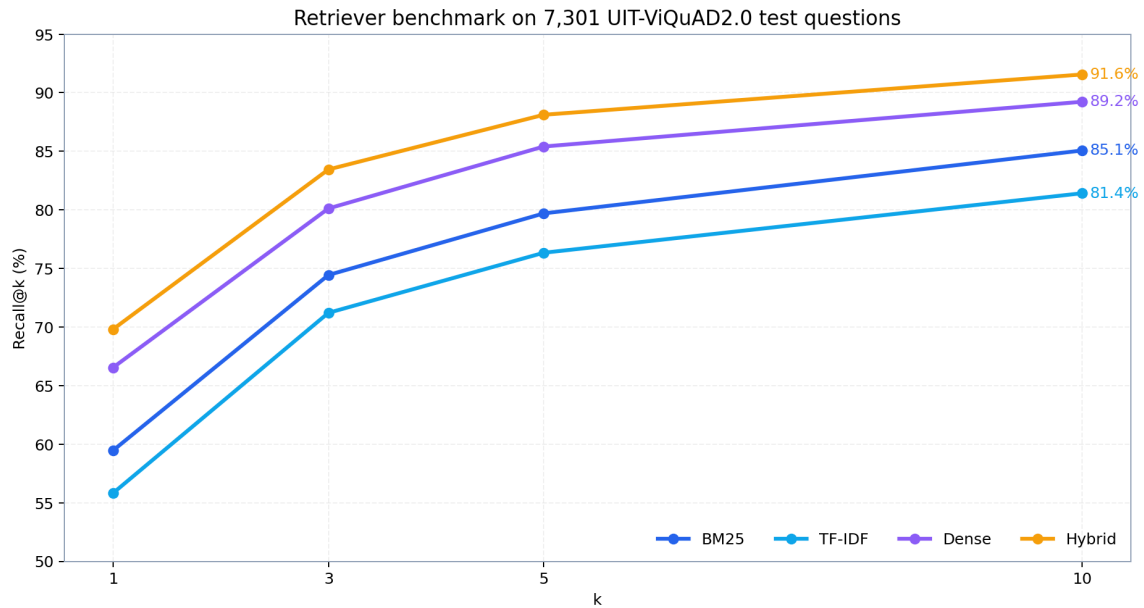


Figure 10.1: Recall@ k từ artifact Retriever hiện tại

Hybrid cải thiện tuyệt đối 6,49 điểm phần trăm Recall@1 và 6,49 điểm Recall@10 so với BM25. Dense có QPS lớn nhất trong artifact vì embedding corpus/index đã được xây trước; con số này không bao gồm cost khởi động model và không nên suy rộng sang API CPU end-to-end.

10.2 Đánh giá Reader PhoBERT

Artifact đầy đủ có 3.814 validation examples. Kết quả trong [Table 10.2](#) and [figure 10.2](#).

Table 10.2: Reader checkpoint đang được cấu hình

Nhóm	Chỉ số	Giá trị
Overall (3.814)	EM / F1	13,87 / 36,25
Answerable (2.653)	EM / F1	0,79 / 32,96
Answerable	predicted-empty rate	34,83%
Unanswerable (1.161)	Accuracy	42,38%
Toàn bộ	predicted-no-answer rate	37,13%

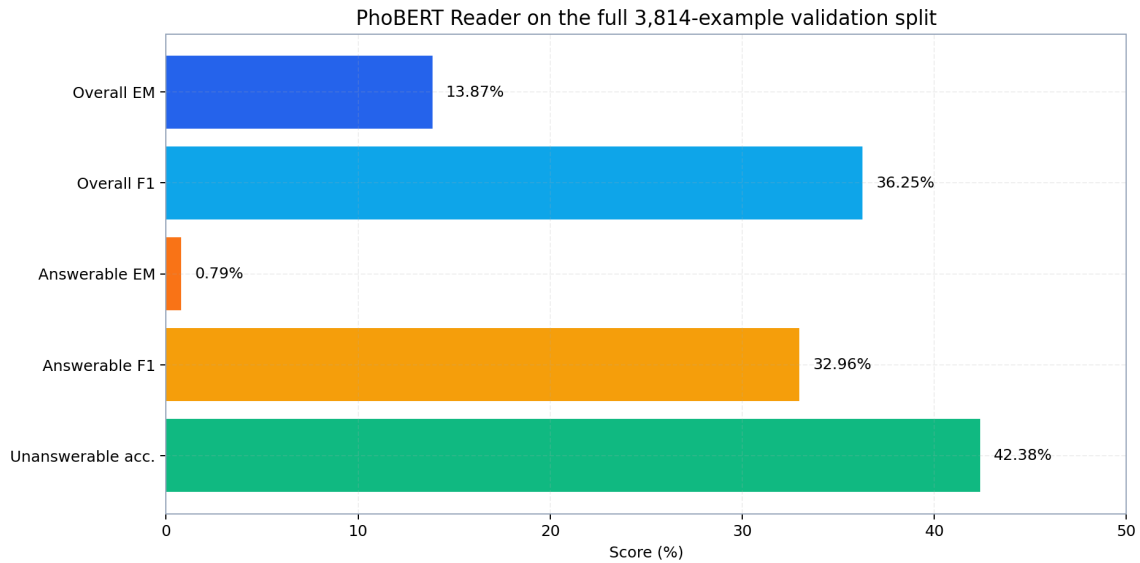


Figure 10.2: Reader PhoBERT trên full validation; nguồn: validation metrics của checkpoint

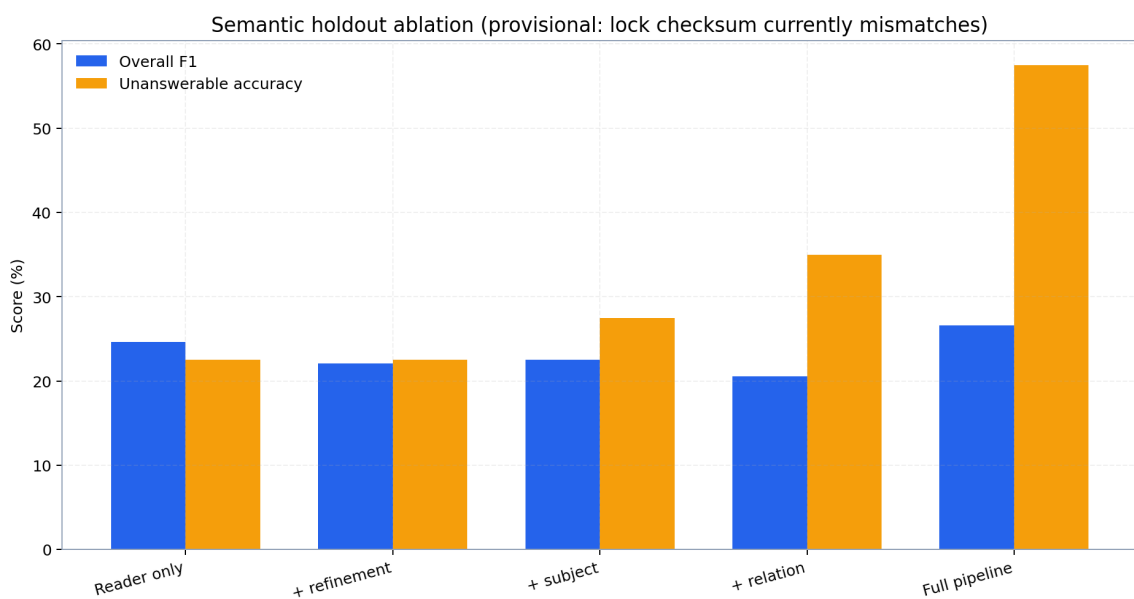
Answerable F1 đã tốt hơn Reader cũ, nhưng Answerable EM rất thấp, cho thấy span còn dài/ngắn lệch chuẩn dù token overlap có ý nghĩa. Deployment decision trong `results/reader/new_checkpoint/deployment_decision.json` ghi rõ `status=experimental`, `production_ready=false`. So với checkpoint trước, Answerable F1 tăng 3,88 điểm và empty rate giảm 8,78 điểm, nhưng Overall F1 giảm 1,78, Unanswerable Accuracy giảm 16,11 và Answerable EM giảm 13,68 điểm. Vì vậy runtime đang dùng checkpoint thử nghiệm, không đồng nghĩa đã được promote theo chuẩn nghiên cứu.

10.3 Semantic holdout và ablation

Artifact v2-final mô tả holdout 260 mẫu, parser relation rate 89,62%, predicate rate 87,31%, subject rate 63,46% và fully parsed rate 57,69%. Diagnostic paraphrase mới đạt 11/12 (91,67%). Full pipeline nâng Unanswerable Accuracy từ 22,50% lên 57,50% và giảm false-positive rate từ 77,50% xuống 42,50%, nhưng false-negative rate tăng từ 15,00% lên 44,09%. Overall F1 tăng từ 24,63 lên 26,56.

Table 10.3: Ablation trên candidate pool cố định của semantic holdout

Cấu hình	Overall F1	Answerable F1	Unans. Acc.	FP / FN rate
Reader only	24,63	25,02	22,50	77,50 / 15,00
+ refinement	22,05	21,97	22,50	77,50 / 17,27
+ subject	22,51	21,14	27,50	72,50 / 25,45
+ relation	20,59	17,97	35,00	65,00 / 31,82
Full pipeline	26,56	20,94	57,50	42,50 / 44,09

**Figure 10.3:** Ablation semantic; kết quả tạm thời do checksum lock đang lệch

Cảnh báo integrity. File lock khai báo SHA-256 bắt đầu bằng c9c4694f..., trong khi test ngày 20/08/2026 tính hash hiện tại bắt đầu bằng df950673.... Trạng thái lock vẫn ghi LOCKED_DO_NOT_TUNE, vì vậy dataset đã thay đổi mà manifest chưa cập nhật hoặc cần điều tra. Cho đến khi khôi phục đúng file/lock và chạy lại, các số liệu ở Table 10.3 and figure 10.3 chỉ là diagnostic tạm thời, không phải kết quả holdout bất biến.

Độ phủ rule. Artifact rule coverage cho thấy cause prefix, location và purpose interrogative đạt precision 1,00 trên lần lượt 31, 18 và 18 match; auxiliary/copula boundary đạt 0,961 và 0,991. Điểm yếu gồm modifier boundary 0,554, death-time marker 0,20 và identity 0/1. Những mẫu có coverage quá thấp không đủ để kết luận khả năng tổng quát.

10.4 Latency semantic

Artifact holdout ghi average 5.371 ms, p50 4.439 ms và maximum 29.607 ms cho pipeline in-process trên host CPU. Đây không phải SLA: phép đo không gồm HTTP serialization, có cold/warm model effects và candidate pool cố định. Frontend timeout ở 60 giây để tránh chờ vô hạn.

10.5 Build và test tại thời điểm viết

Table 10.4: Kết quả kiểm tra ngày 20/08/2026

Lệnh	Kết quả	Ghi chú
<code>npm run build</code>	Thành công	3.351 module; cảnh báo hai JS chunk lớn hơn 500 kB
<code>python -m pytest -q</code>	178 passed, 10 failed	188 test; 7 deprecation warnings; 147,51 giây

Mười failure không bị che giấu. Chúng thuộc các nhóm:

- semantic false premise/cause và thứ tự rejection reason;
- question-type regression cho alias, location/entity và câu hỏi mô tả số lượng;
- fallback span dài hơn expected;
- hai test vẫn kỳ vọng reader weight 0,50 trong khi config hiện là 0,60;
- semantic holdout checksum không khớp lock.

Do có failure semantic thật, trạng thái kiểm thử tổng thể là **chưa xanh**. Hai failure trọng số có thể là test stale, nhưng phải được quyết định và cập nhật có chủ đích; không nên tự coi là pass.

10.6 Kiểm chứng source title

Các test source-title và tích hợp QA nằm trong phần đã pass. Runtime kiểm tra câu “Phạm Văn Đồng là ai?” trả source title “Phạm Văn Đồng” trong khi passage vẫn giữ đủ ngày sinh–mất. Điều này xác nhận lỗi title bị cắt thô đã được sửa ở backend, không chỉ bằng CSS.

Chapter 11

Thảo luận, giới hạn và lộ trình

11.1 Điểm mạnh hiện tại

- Pipeline có bằng chứng end-to-end và không sinh câu trả lời ngoài passage.
- Hybrid Retriever đạt Recall cao nhất trong benchmark hiện tại.
- Semantic policy có version, gate order và rejection reason có thể audit.
- Socratic follow-up cũng grounded; không đưa thêm LLM khó kiểm soát.
- Feedback không tự học, có validation, dedup, conflict và review lifecycle.
- UI hội thoại tách phần người dùng cần khỏi debug/pipeline disclosure.

11.2 Giới hạn quan trọng

1. **Reader chưa đủ mạnh:** Answerable EM 0,79 và checkpoint vẫn experimental.
2. **Semantic trade-off:** giảm false positive nhưng làm false negative tăng.
3. **Holdout mất tính bất biến:** checksum mismatch làm diagnostic semantic chưa thể dùng như benchmark khóa.
4. **Benchmark/runtime lệch tầng:** offline document-level và online passage-level có scorer/boost khác nhau.
5. **Evaluation dashboard drift:** Reader artifact cũ và giá trị latency/error tính chưa đại diện runtime.
6. **Feedback chưa có dữ liệu thật:** analytics mới chứng minh plumbing.

7. **Bảo mật:** review/document endpoints chưa có auth, audit actor hay rate limit.
8. **Hiệu năng:** Reader CPU và bundle Mari lớn; build cảnh báo code splitting.
9. **Đánh giá Socratic:** chưa có human study hoặc metric su phạm.

11.3 Lộ trình ưu tiên

Table 11.1: Backlog đề xuất

Mức	Hạng mục	Hành động	Tiêu chí hoàn tất
P0	Holdout integrity	Khôi phục đúng JSONL hoặc tạo holdout v2 mới; cập nhật lock rồi chạy lại	checksum test pass, provenance đầy đủ
P0	Semantic regressions	Sửa false-premise/cause, type classifier, fallback boundary và rejection priority	toàn bộ test semantic/pipeline pass
P0	Evaluation source	Endpoint đọc đúng reader profile/metrics của checkpoint active; bỏ hard-code	UI và artifact khớp tuyệt đối
P1	Reader	Audit span label, retrain/checkpoint select bằng Answerable F1 và boundary-aware EM	full 3.814 validation, không giảm safety floor
P1	End-to-end eval	Tạo gold passage và answer độc lập, passage-level benchmark giống runtime	báo cáo EM, F1, abstention, latency có version
P1	Feedback security	Authentication, RBAC, actor audit, CSRF, rate limit và backup	review endpoint không public
P1	Feedback loop	Thu dữ liệu thật, review, export, train offline và benchmark đối chứng	promotion có approval và rollback
P2	Performance	Worker/model cache, batch queue, lazy chunk splitting cho Three.js	giảm cold start và bundle warning

Mức	Hạng mục	Hành động	Tiêu chí hoàn tất
P2	Socratic UX	Rubric chuyên gia và study người dùng	usefulness, grounding và duplication được đo

11.4 Kết luận

VIQA Nexus đã vượt khỏi demo Retriever–Reader đơn giản để trở thành prototype QA có truy vết nguồn, nhiều chiến lược truy hồi, candidate refinement, semantic gating, Socratic tutoring, giao diện hội thoại và feedback review. Kết quả Retriever cho thấy Hybrid có lợi rõ ràng, trong khi Reader và semantic layer vẫn là nút thắt chính.

Giá trị quan trọng của phiên bản hiện tại không chỉ nằm ở một con số F1 mà ở khả năng nhìn thấy quyết định: passage nào được truy hồi, span nào bị loại, gate nào thất bại, feedback nào đang chờ duyệt và model/corpus/policy nào tạo câu trả lời. Tuy nhiên, prototype chưa sẵn sàng để được gọi là production: test chưa xanh, holdout checksum đang lệch và dashboard đánh giá chưa đồng bộ artifact. Lộ trình hợp lý là khôi phục tính tái lập trước, sửa regression, rồi mới tối ưu Reader và mở vòng lặp dữ liệu thật.

Appendix A

Hướng dẫn chạy và tái lập

A.1 Chạy hệ thống

Trên Windows có thể chạy `run_system.bat`. Hoặc tách hai terminal:

```
python backend/viqa_api.py
npm run dev
```

Sau đó mở <http://localhost:5173/>. API health ở <http://localhost:8000/health>. Đường dẫn /HTTP không tồn tại và sẽ trả 404; URL frontend đúng kết thúc ở dấu gạch chéo hoặc route hợp lệ.

A.2 Build và kiểm thử

```
npm run build
python -m pytest -q
python evaluate_socratic.py
python evaluate_feedback_loop.py
python evaluate_semantic_holdout.py --help
```

Khi tái lập semantic benchmark phải kiểm tra checksum lock trước khi đọc metric.

A.3 Mẫu request QA

```
POST /api/ask
```

```
Content-Type: application/json
```

```
{"question": "Pham Van Dong la ai?", "retriever": "bm25",  
  "reader": "phobert", "top_k": 10}
```


Appendix B

Mười câu hỏi demo

Danh sách dưới đây bao phủ identity, role, time, location, cause, purpose và entity; không phải mọi câu đều được bảo đảm có gold answer trong corpus hiện tại.

1. Phạm Văn Đồng là ai?
2. Phạm Văn Đồng sinh vào thời gian nào?
3. Phạm Văn Đồng từng giữ những chức vụ nào?
4. Hội nghị Genève năm 1954 có ý nghĩa gì đối với Đông Dương?
5. Cách mạng Pháp diễn ra ở đâu?
6. Vì sao Madame du Barry bị khinh miệt?
7. Công trình nào nằm trên đỉnh Montmartre?
8. Thực vật có hoa được chia thành những nhóm nào?
9. Mục đích của việc thành lập một tổ chức được nêu trong tài liệu là gì?
10. Sự kiện nào gắn với nhân vật đang được đề cập trong passage nguồn?

Appendix C

Ma trận truy vết báo cáo—mã nguồn

Table C.1: Nguồn kiểm chứng cho các nội dung chính

Nội dung	File chính
QA config	config/qa_pipeline.json, backend/config.py
Semantic policy/gates	config/semantic_policy.json, reader/candidate_validation.py
Question semantics	reader/question_semantics.py, reader/relation_validator.py
Chunking/source title	backend/chunking.py, backend/source_titles.py
Online QA/RRF/API	backend/viqa_api.py
Socratic	backend/socratic.py, config/socratic.json
Feedback/analytics	backend/feedback.py, backend/feedback_analytics.py
Frontend chat	src/pages/HomePage.tsx, src/components/chat/AssistantMessage.tsx
Composer keyboard	src/components/chat/ChatComposer.tsx
Knowledge Blind Spots	src/pages/KnowledgeBlindSpotsPage.tsx
Retriever metrics	results/retriever_eval_all.json
Reader metrics/profile	results/reader/heidiie_phobert_finetuned_viquad_full/
Semantic diagnostics	results/semantic_holdout_v1_report_refactored_v2_ final.json
Test status	lệnh python -m pytest -q, ngày 20/08/2026

Tài liệu tham khảo

- [1] D. Chen, A. Fisch, J. Weston, and A. Bordes. Reading Wikipedia to Answer Open-Domain Questions. In *Proceedings of ACL*, 2017.
- [2] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009.
- [3] G. Cormack, C. Clarke, and S. Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of SIGIR*, 2009.
- [4] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, 2019.
- [6] Y. Liu et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019.
- [7] D. Q. Nguyen and A. T. Nguyen. PhoBERT: Pre-trained Language Models for Vietnamese. In *Findings of EMNLP*, 2020.
- [8] K. V. Nguyen et al. UIT-ViQuAD 2.0: A Vietnamese Dataset for Reading Comprehension with Unanswerable Questions. In *Proceedings of LREC*, 2022.
- [9] B. Settles. Active Learning Literature Survey. University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.